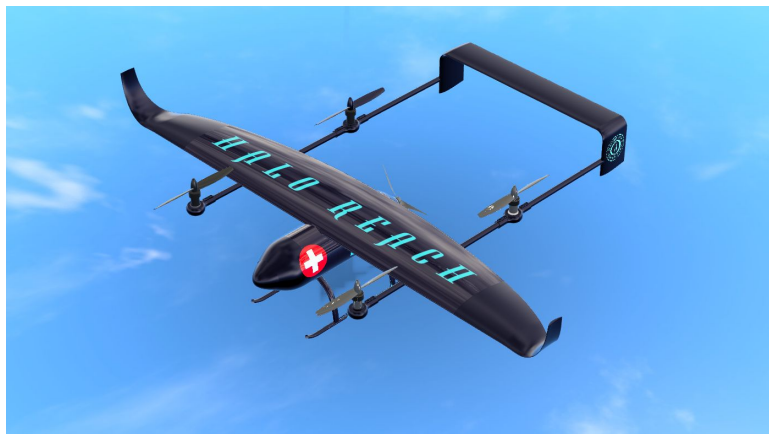

HALO REACH: High-Fidelity Simulation of Fixed-Wing VTOL UAV using Gazebo

UAS for Disaster Relief - AE3 2024



Student: Jeongheon Lee

CID: 01754474

Supervisor: Dr. Sophie Armanini

Department: Department of Aeronautics

Course: MEng Aeronautical Engineering

Module: AERO60004 - Group Design Project

Date: 17/06/2024

Academic Year: 2023-2024

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ
U.K.



Table 1. People involved in HALO REACH - GDP (**student in bold**)

Team code	Team	Names	CID
FATT01	Team Leads	Moreno, Patricia	01857449
		Rashid, Hasan H	02020313
FATT02	Aerodynamics	Alam, Shahriar	02037395
		Dirirsa, Naboth	02026211
		Lawlor, Max F	02026199
		Singh, Parasdeep	02099678
		Homa, Matthew M	02038403
FATT03	Structures	Leng, Jiaxun (Aaron)	02015060
		Wang, Zhiyan	01911209
		Liu, Stephen	02015258
		Chauhan, Shivani	02096006
FATT04	Systems and propulsion	Lin, Yongkai	02029838
		Chursuk, Tharm	02048996
		Wankhade, Divij P	02027906
		Aggarwal, Ishaan K	02030969
		Halim, George	02059571
FATT05	Flight dynamics	Baikady, Thilak S	02016794
		Scott-Brown, Ruairidh	01734080
		Allen, Harry	02028116
		Tang, Shun S	02023280
		Lee, Jeongheon (Edward)	01754474
FATT06	Add-ons	Rosenthal, Adam T	02027024
		Ramteke, Shreya	02079050
		Nakhla, Andrew	02044646

Abstract

This report covers the development process of the high-fidelity simulation environment, which can test the controllers of a fixed-wing UAV. Gazebo simulator was chosen due to its availability and flexibility for this project. The model with the same inertial and visual properties as the actual vehicle design could be used using the primary design parameters. Then, the aerodynamic properties of the lifting surfaces used in the vehicle were modelled using the plugin, which generates aerodynamic forces and moments during the simulation. The longitudinal controller was tested using the developed simulation environment. The testing result showed that the longitudinal controller operates as expected and shows a certain amount of performance in altitude tracking. However, the unexpected momentum loss made testing the speed controller inside the longitudinal controller impossible. The analysis after the testing demonstrated that the discontinuity in linear momentum and the performance mismatch between the simulated pusher propeller thrust and expected thrust could be the primary cause.

Contents

Table of Contents	iii
List of Figures	iv
List of Tables	v
Nomenclature	vi
1 Introduction	1
2 High-Fidelity Simulation Development	1
2.1 Simulator Selection	1
2.2 Simulation Flowchart	2
2.3 Vehicle Modelling	2
2.4 Aerodynamics Plugin	4
2.4.1 Plugin Overview	4
2.4.2 Main Wing, Empennage, and Control Surfaces	4
2.4.3 Propellers	6
3 Controller Testing and Performance Prediction	7
3.1 Longitudinal Controller Testing Setup	7
3.2 Testing Process	8
4 Discussion and Conclusion	10
4.1 Limitations of Simulation Environment and Model	10
4.2 Conclusion	12
References	13
A Drag Polar and Momentum Curve of the Used Aerofoils	15
B The code for Gazebo Simulation - Model and Liftdragplugin	17
B.1 Model Code	17
B.2 Liftdragplugin Code	27

List of Figures

1	Simulation flowchart	2
2	The spawned model and its moment of inertia	3
3	Liftdragplugin applied to the right section of the main wing	4
4	Lift polar interpolation results of the fixed surfaces	5
5	Plot of Effectiveness parameter vs. chord ratio [25]	5
6	Lift polar interpolation results of the propellers	7
7	Controller testing environment	7
8	First controller test result - Original Configuration	8
9	Second controller test result - Doubled thrust	9
10	Third controller test result - Constant Velocity	10
11	Velocity and acceleration generated by pusher propeller at maximum RPM	11
12	NACA-6409	15
13	NACA-0009	15
14	Clark-Y	16
15	NACA-23012	16
16	E216	16

List of Tables

1	People involved in HALO REACH - GDP (student in bold)	i
2	Parameters used for the fixed lifting surfaces	6
3	Propeller blade profiles and the used conditions for optimisation [29] [27]	6
4	Parameters used for the propellers	6
5	Controller Testing Condition	8
6	First controller test: Drag force generated by Liftdragplugin and the required drag force	10

Nomenclature

α	Anlge of Attack (rad)
δ	Control surface deflection angle (rad)
Φ	Roll angle
Ψ	Yaw angle
ρ	Flow density (Air = 1.22 kg/m^3)
τ	Effectiveness Parameter
Θ	Pitch angle
c	Chord length (m)
C_d	Drag coefficient
C_L	Lift coefficient
C_M	Moment coefficient
m	Mass (kg)
N	Normal force (N)
U_∞	Incoming flow velocity (m/s)
A	Reference area
AoA	Angle of Attack
HALO REACH	Highly Aerodynamic Light Operation Remote Emergency Aid & Crisis Help
MTOW	Maximum Takeoff Weight
UAS	Uncrewed Aerial System
VTOL	Vertical Take-Off and Landing
x	x-coordinate position
y	y-coordinate position
z	z-coordinate position

1 Introduction

HALO REACH is the Unmanned Aerial System (UAS) specifically designed for operation in disaster sites. Fixed-wing VTOL design was selected to achieve adequate range and endurance while keeping the advantage of manoeuvrability of VTOL system [1]. Since the vehicle uses different actuators during VTOL and fixed-wing flight regimes, the development of the VTOL and Fixed-wing can be done separately using a different dynamic model [2]. The Flight Dynamics and Control Systems sub-team of HALO REACH, therefore, decided to develop the VTOL, fixed-wing, and transition controller separately [3] [4] [5].

While each controller is designed using its own dynamic model, the linearised state-space model is often used as a primary model to design and tune the controller [6]. However, as the linearised model always has a certain degree of imperfection, the designed controllers are required to go through a real-world test. However, for a large and complex system, such as an aerial vehicle, the real-world test of a newly developed controller becomes a risky mission. As a result, in aerospace industries, simulation has been extensively used between controller development and a real-world test [7]. In recent years, along with the growth in the UAV industry, simulation has begun to be used in the development of UAVs as well [8]. The Flight Dynamics and Control Systems sub-team also decided to use a high-fidelity simulation to check whether the developed controllers operate accordingly when integrated and do the secondary tuning of the controller based on the simulation test result.

This report will explain the development process of a high-fidelity simulation environment which is designed to test HALO REACH specifically. Then the longitudinal controller for the fixed wing mode will be tested not only to check the controller performance but also to check the simulation environment operates as expected. Based on the test result, the error and limitation exist in the simulation environment will be identified and analysed.

2 High-Fidelity Simulation Development

2.1 Simulator Selection

The appropriate simulator had to be selected first. To make sure the simulation can replicate the real-world situation as much as possible, the simulator had to be equipped with a good physics model. Since the primary goal of this simulation is to verify the designed controllers, the simulator should be capable of subscribing the actuator inputs and applying them to the appropriate motors and control surfaces. Lastly, a simulator itself should publish the vehicle states so that the controllers' feedback loops are tested appropriately.

Based on the required capabilities of the simulator, two options were primarily considered: Simulink and Gazebo. Simulink was considered first since it was the platform that was going to be used for controller development. There was a huge advantage in sending and receiving the commands and states as they could be done 'seamlessly.' At the same time, several blocks that were specifically designed for simulating the UAV were already available, which would make the simulation development simpler. However, several limitations of Simulink could not be disregarded. The simulation of the aerial vehicle in Simulink is based on solving the reduced model of the vehicle and sending the vehicle state information to the 3D visualization block [9]. This method can yield a relatively accurate result only when the vehicle shows a small deviation from its equilibrium state. Besides, the provided reduced-model block only supports quadcopter and fixed-wing UAVs, which introduces another complexity to make it work for a fixed-wing VTOL model.

Gazebo, an open-source simulator, has been developed to provide the simulation environment for numerous hardware and sensors[10]. Its main field of application is robotics, but it is still widely used for the simulation of UAVs. For instance, the Defense Advanced Research Projects Agency (DARPA) launched 'The Swarm Challenge' in 2017, whose main goal was to develop effective tactics for capture the flag game done by a swarm of UAVs, and the Gazebo was designated as a simulation environment [11]. The main advantage of Gazebo is its flexibility. The user can create any model and put it inside the simulation environment, even without any information on the model's state space. Instead, Gazebo

utilizes various physics engines, such as DART, to capture the behaviour of the model in real time [10]. The availability of an aerodynamics plugin, which enables defining different aerodynamic properties at each part of the vehicle, is another big advantage of Gazebo [12]. Still, due to its flexibility, the development of a simulation environment based on Gazebo requires lots of hard coding, especially when building the model and setting up the physical environment.

Gazebo was eventually chosen as a simulator for several reasons. First, throughout the project, the vehicle design process and the simulation environment development had to be done simultaneously due to time constraints. With Gazebo, a simulation environment could be developed only with the information on the baseline configuration of the vehicle, while the vehicle's parameters could be applied later. Second, since the simulation was planned to be used as a final step of verifying the developed controllers, capturing the accurate behaviour of the vehicle under its control input was crucial, and the application of a real-time physics engine would be more advantageous than just using the state space of the vehicle. Lastly, Gazebo also supports the simulation of various sensors, which would give an opportunity to test the controller and any autopilot algorithm together in the later phase of vehicle development.

2.2 Simulation Flowchart

The connection between Gazebo and the controller had to be established to test the controller. As mentioned in the previous section, each controller component was planned to be developed using MATLAB and Simulink. Due to its wide application in the Robotics industry, Gazebo is often used along with Robot Operating System (ROS), which is a middleware that connects a robot's software and hardware. In this project, The use of ROS brings a considerable advantage when the actual hardware model is operated based on ROS, which makes the transition between the simulation and real-world testing seamless. In this project, however, the manufacturing of the actual vehicle is not included, which makes the use of ROS unnecessary. As a result, a direct connection between Simulink and Gazebo was decided to be used in this project.

The connection between Simulink and Gazebo was established by the existing plugin provided by Simulink [13]. Cosimplugin uses TCP/IP protocol, which connects two different computers using an IP address and transports the data through a TCP port. As the protocol requires at least two different computers, Gazebo was installed and run on the virtual computer, while the Simulink was operated on the main computer. The diagram of the simulation is illustrated in Figure 1.

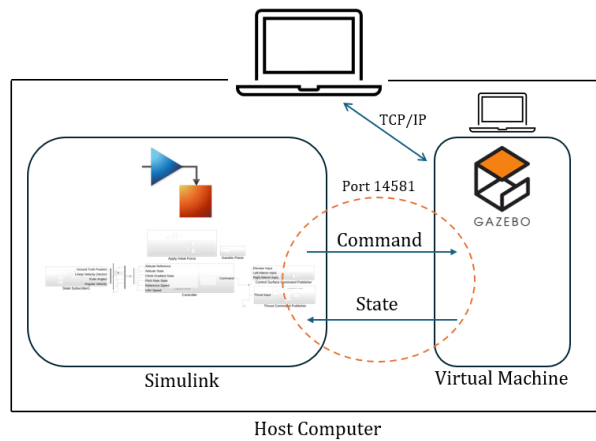


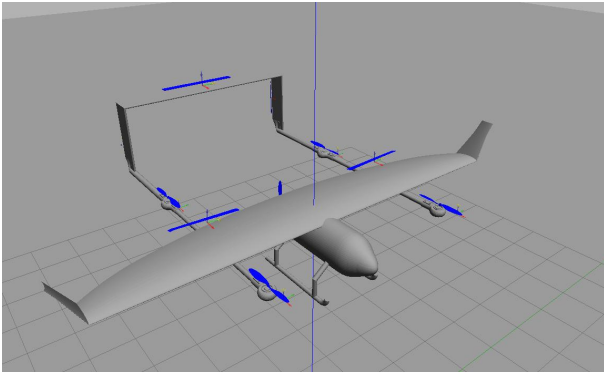
Figure 1. Simulation flowchart

2.3 Vehicle Modelling

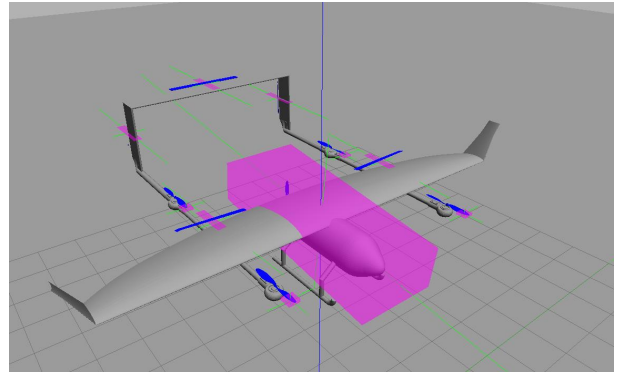
The designed vehicle had to be modelled and spawned inside the simulation environment. Simulation Description Format (SDF) is adopted by Gazebo to define the physical and visual properties of the model inside the simulation environment [14]. The structure of the existing SDF file that can be used

to generate a fixed-wing VTOL vehicle was imported, and the model of HALO REACH for Gazebo simulation could be created based on that. The structure was imported from the SDF file of 'Alti Transition' created by Ardupilot [15] [16].

The model consists of multiple links, which define the inertial, collision, and visual properties of each part [17]. The used links in the fixed-wing VTOL model can be categorised into five types: base, payload, motor/propeller, sensor, and control surfaces. The base link reflects the inertial property of the vehicle when the motors and propellers are uninstalled. The moment of inertia of each link can be defined using the nine inertia matrix elements. Each element of the base inertia tensor was calculated by considering each base component as a point mass at its own centre of gravity. The payload was created as a separate link to make the testing with and without the payload easier. However, since the CG of the payload is considerably close to the CG of the base, the inertia tensor of the payload was set to zero. The inertial information of the four VTOL motor/propeller combinations was derived by assuming each motor was a cylinder with a uniform density. In the case of the pusher propeller, only the link of the propeller itself had to be defined as the engine's inertial information is included in the base link. Lastly, both the sensor and control surface links were defined as a point mass as their mass is relatively small compared to the vehicle's total mass.



(a) Spawned Model



(b) Applied moment of inertia

Figure 2. The spawned model and its moment of inertia

The motors and control surfaces had to be separated from the base link to make them controllable. Along with defining the link for the motors and control surfaces, the joint element, which defines how the two links are connected to each other, had to be defined in a specific way to simulate the actuation. Among various types of joints, 'revolute' allows the child link to have one rotational degree of freedom. Both the motor and control surface links were, therefore, connected to the base link using a revolute joint to control the rotational speed and deflection angle at the later phase of the project. In contrast, as both the IMU sensor and payload should maintain their position on the vehicle, those were connected to the base link using a 'fixed' joint.

'Collision' element should define the shape of each link. The collision of the link can be set as one or more simple shapes included in the API of SDF, or it can be directly imported from the CAD model of the component. For the base link, the CAD model of the vehicle without motors and control surfaces was imported [18]. The simple cylinder shape was used for the four VTOL motor/propeller combinations. The radius and height of the cylinder were defined as the propeller disk radius and the total height of the motor and the propeller. The collision of the pusher propeller, on the other hand, could be created by just considering the propeller disk radius and height.

How each link is visualised can be defined separately using 'visual' element. The CAD models were used to visualize the base, propeller, and control surface links. The implementation of CAD models did not only improve the appearance of the simulation but also make the observation of the actuators easier. However, the visual element plays no role in the dynamics of the model as it is already defined by the link and collision element. The base link could reuse the CAD model of the vehicle which was used for collision element [18]. To visualize the operation of actuators, the CAD model of elevator,

rudder, propeller, and aileron from 'Alti Transition' model was inherited but scaled up [15] [16].

2.4 Aerodynamics Plugin

The lift, drag, and moment generated from the lifting surfaces of the vehicle can be generated by defining the aerodynamic properties of each surface using Liftdragplugin. This fixed wing VTOL UAS features the three types of lifting surface, which are main wing, control surfaces, and propellers. Each lifting surface type is given its aerodynamic property based on the certain method which fits to its characteristics.

2.4.1 Plugin Overview

The general structure of the code used to define the aerodynamic property of one lifting surface is shown in Figure 3. Liftdragplugin available in Gazebo heavily relies on thin aerofoil theory, which assumes the direct relationship between the angle of attack (AoA) and the aerodynamic coefficients [19] [20]. However, the major problem of thin aerofoil theory is that it cannot reflect the sudden change in aerodynamic coefficients after the AoA of an aerofoil exceeds its stall AoA. To overcome this problem, Liftdragplugin uses two slope values for each coefficient. One of them is used to generate the curve (i.e. polar) before the stall angle, and the remaining one is used for the curve after the stall angle. The plugin accepts only one numerical value to define the curve. Therefore, the final curve becomes a non-smooth plot linearised before and after the stall angle. Lastly, the plugin does not receive any y-intercept value, so any lifting surface which uses a non-symmetric aerofoil should shift the graph by offsetting the initial AoA.

```

1  <plugin filename="libLiftDragPlugin.so"
2    name="Right_MainWing_LiftDrag">
3    <a0>0.075</a0> <!-- Initial AoA. Zero Lift AoA needs to be considered -->
4    <cla>4.74394949633103</cla> <!-- dCL/dAlpha slope -->
5    <cda>0.0684</cda> <!-- dCd/dAlpha slope -->
6    <cma>-0.2172</cma> <!-- dCM/dAlpha slope -->
7    <alpha_stall>0.23247786</alpha_stall> <!-- Stall angle -->
8    <cla_stall>-0.8529</cla_stall> <!-- dCL/dAlpha slope after stall -->
9    <cda_stall>0.8918</cda_stall> <!-- dCd/dAlpha slope after stall -->
10   <cma_stall>-0.2172</cma_stall> <!-- dCM/dAlpha slope after stall -->
11   <cp>0.170968049 -1.5 0.165047</cp> <!-- Centre of Pressure -->
12   <area>2.1175</area> <!-- Reference Area -->
13   <air_density>1.2041</air_density>
14   <forward>1 0 0</forward> <!-- Lifting Surface forward direction -->
15   <upward>0 0 1</upward> <!-- Lifting Surface upward direction -->
16   <link_name>Base</link_name> <!-- The lifting surface link -->
17   <control_joint_name>right_aileron_joint</control_joint_name> <!-- control surface joint -->
18   <control_joint_rad_to_cl>-2.143</control_joint_rad_to_cl> <!-- dCL/dDelta -->
19 </plugin>

```

Figure 3. Liftdragplugin applied to the right section of the main wing

2.4.2 Main Wing, Empennage, and Control Surfaces

There are several methods which allow the generation of polar of a specific lifting surface, including experiment and physics-based approach [21]. However, learning and using those methods were considered not to be included in the scope of this project. Instead, the polar data were imported from the UIUC Airfoil Coordinates Database for each lifting surface [22]. The main wing uses NACA 6409 aerofoil, and the expected Reynolds number was in the order of millions [23]. In the case of empennage, NACA 0009 aerofoil was used for both horizontal and vertical tailplane [24]. The linearization

results of the lift coefficient are shown in Figure 4. The interpolation plots of the drag and moment coefficients are available in the Appendix.

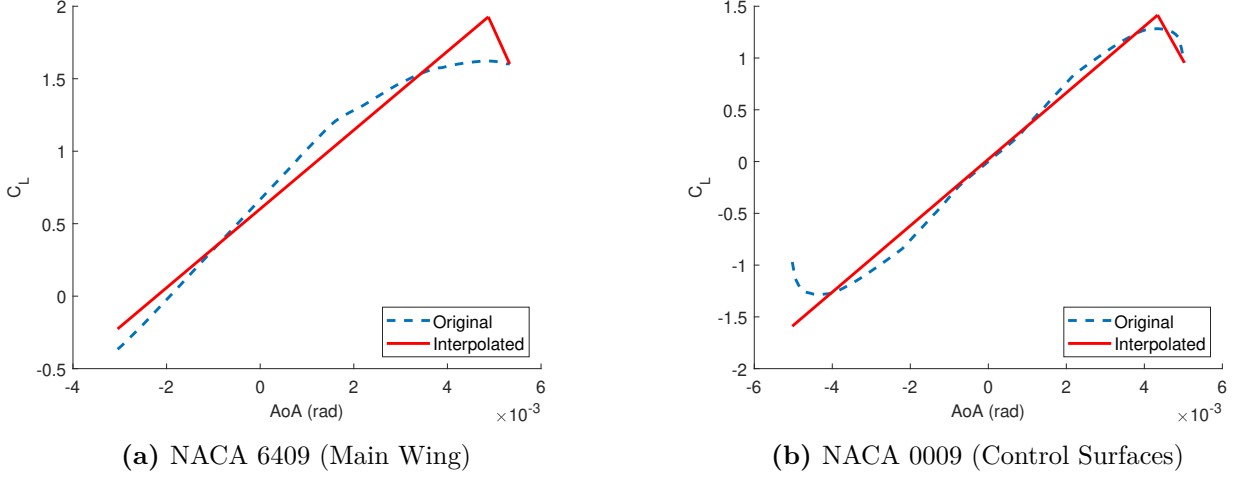


Figure 4. Lift polar interpolation results of the fixed surfaces

The lift generated on each lifting surface should be allowed to be controlled with the actuation of the control surfaces: aileron, elevator, and rudder. To simulate this, the plugin uses the relationship between the control surface's deflection angle and the lifting surface's overall lift coefficient, as stated in Equation (1).

$$C_{L_{total}} = \frac{dC_L}{d\alpha} \alpha_{\infty} + \frac{dC_L}{d\delta} \delta \quad (1)$$

According strip integration method, the change in lift coefficient due to the deflection of control surfaces can be calculated based on the effectiveness parameter τ , which represents the change in AoA of the lifting surface due to the deflection of a control surface.

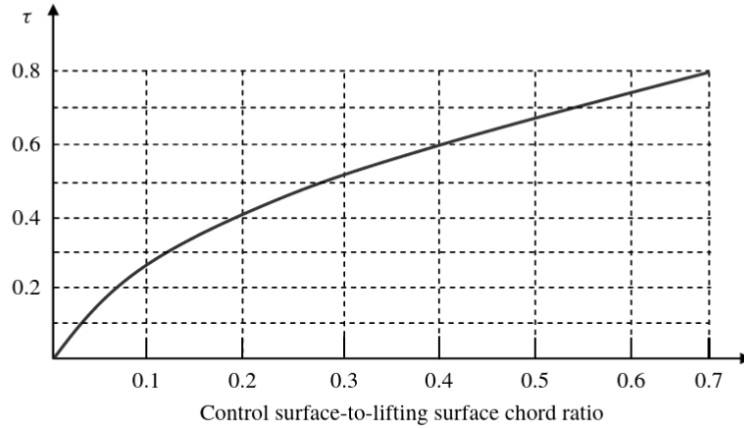


Figure 5. Plot of Effectiveness parameter vs. chord ratio [25]

By finding the value of τ using a chord ratio and Figure 5, the change in lift coefficient due to the control surface deflection can be expressed as shown in equation (2) [25].

$$\frac{dC_L}{d\delta} = \frac{dC_L}{d\alpha} \tau \quad (2)$$

The final aerodynamic parameters used for the main wing and control surfaces are shown in Table 2.

Table 2. Parameters used for the fixed lifting surfaces

Lifting surface	Used Aerofoil	Slope Values (before/after stall)			Control surface	$dC_L/d\delta$
		$dC_L/d\alpha$	$dC_d/d\alpha$	$dC_M/d\alpha$		
Main wing	NACA-6409	4.74/-0.85	0.07/0.89	-0.22/-0.22	Aileron	-2.14
Empennage	NACA-0009	5.60/-6.66	0.03/1.94	-0.07/1.01	Elevator, Rudder	-3.0

2.4.3 Propellers

During the initial vehicle modelling, the thrust generated by each propeller is usually modelled using the provided thrust coefficient. In the simulation, however, the motor command, given the rotation speed, should be able to generate the thrust. Therefore, the aerodynamics of each propeller blade had to be modelled as the wings and control surfaces.

The propeller blade design is often optimised to achieve good efficiency [26]. The optimisation process usually results in the varying twist angle and chord length along the span of the blade. However, as LiftDragPlugin can utilise only one AoA and set of coefficients from the polars, the process that can simplify the aerodynamic property of the blade while resulting in a similar performance as the original design had to be proposed.

The blades were optimised based on a certain in-flow speed and the motor rotational speed, as shown in Table 3. The simplification process first found the velocity profile at the half-point of the wing, and the twist angle at which the used aerofoil can produce the maximum lift force was identified from the lift polar. The process stated above was adequate to apply LiftDragPlugin when the single aerofoil profile was used for the blade. However, in the case of the VTOL blade, the design result incorporates more than one aerofoil design, which requires another process [27]. The total lift force generated by the propeller blade using n aerofoil profiles can be expressed as Equation (3) [28]. Then, the multiple lift coefficients can be blended into a single value using Equation (4), whose result is shown in 6 (b). The same method was used to derive the drag coefficient of the blade. The final aerodynamic parameters used for the propellers are shown in Table 4.

$$L_{total} = \frac{1}{2} C_{L_{total}} \rho U_{\infty}^2 A_{total} = \sum_{i=1}^n \frac{1}{2} C_{L_i} \rho U_{\infty}^2 A_i \quad (3)$$

$$C_L = \sum_{i=1}^n C_{L_i} \frac{A_i}{A_{total}} \quad (4)$$

Table 3. Propeller blade profiles and the used conditions for optimisation [29] [27]

Blade Type	Used Aerofoil	Flow Velocity (m/s)	Motor RPM	Blade Length (m)
VTOL	NACA-23012	1.0	2500	0.5
	Clark-Y			
Pusher	E216	27.6	2000	0.44

Table 4. Parameters used for the propellers

Blade Type	Slope Values (before/after stall)		
	$dC_L/d\alpha$	$dC_d/d\alpha$	$dC_M/d\alpha$
VTOL	5.88/-3.43	0.09/1.39	-0.0071/0.49
Pusher	5.53/-1.47	0.03/1.05	-0.0045/-0.050

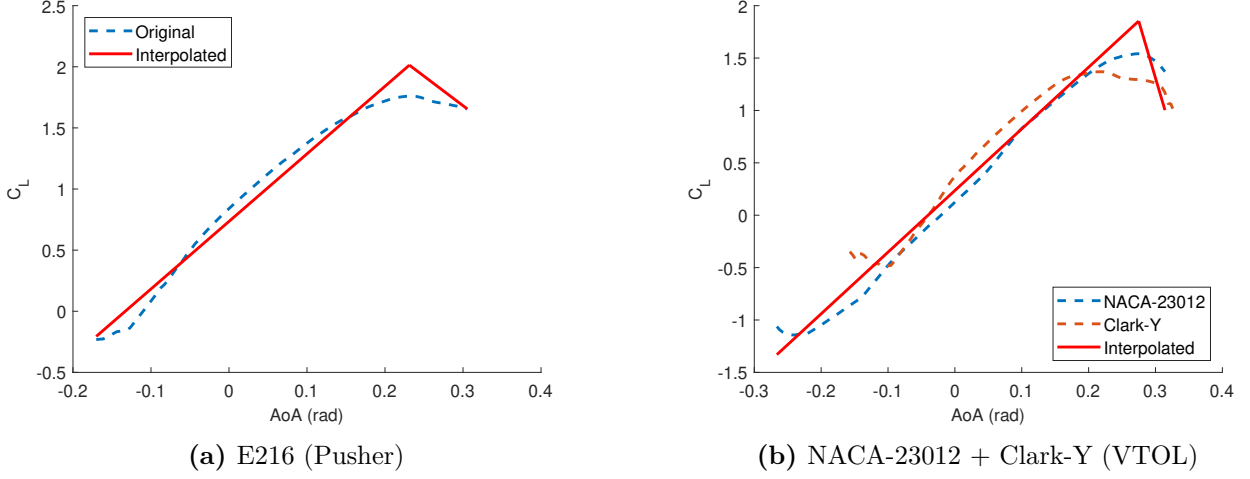


Figure 6. Lift polar interpolation results of the propellers

3 Controller Testing and Performance Prediction

3.1 Longitudinal Controller Testing Setup

The longitudinal controller was available for testing based on the developed Gazebo simulation. As the controller is designed to operate only when the fixed lifting surfaces and pusher propellers are used, the initial condition of the vehicle had to be set to the cruising state when the controller started working. It could be done by exerting force in the x and z directions on the vehicle model for the first three seconds of the simulation to accelerate the aircraft up in the air and generate a sufficient amount of lift. At the same time, to prevent an unexpected directional and lateral motion of the aircraft, the yawing and rolling of the vehicle were locked throughout the testing process.

The longitudinal controller receives altitude and speed setpoints and outputs the thrust value and elevator deflection angle. The four feedback loops are used within the controller, and the altitude, speed, pitch rate, and pitch angle state need to be given to the controller. While the speed and altitude of the vehicle were available by calling `GetVelocity()` and `GetPosition()` commands inside Gazebo, the pitch rate and angle of the vehicle were derived from the IMU sensor model. The thrust command ranges between -0.6 and 0.4, while the elevator command ranges between -1 and 1 [3]. The command values were converted into the angular speed of the motor and the deflection angle of the elevator and published to Gazebo using `setJointVelocity()` and `setJointPosition()` commands, respectively [13].

The longitudinal controller being transported to the testing environment is shown in Figure 7.

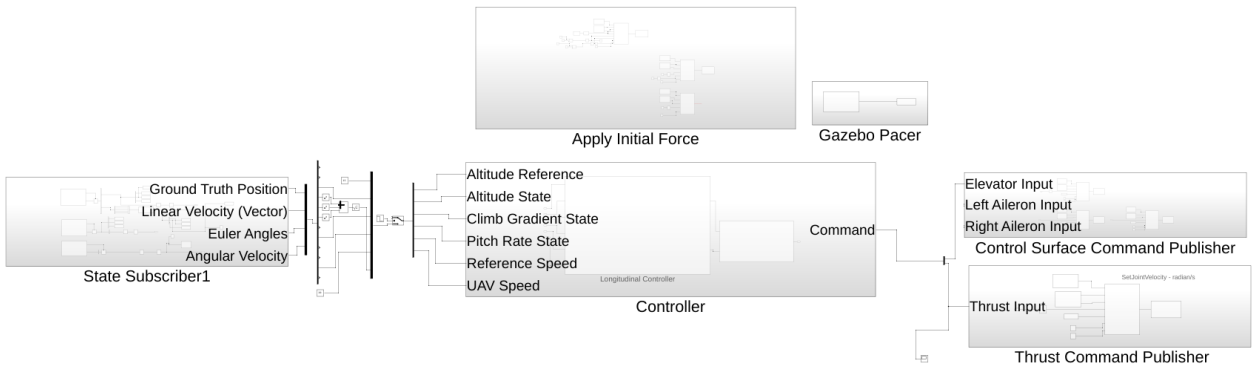


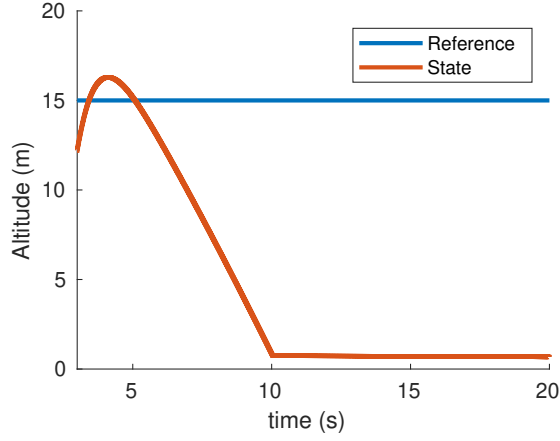
Figure 7. Controller testing environment

3.2 Testing Process

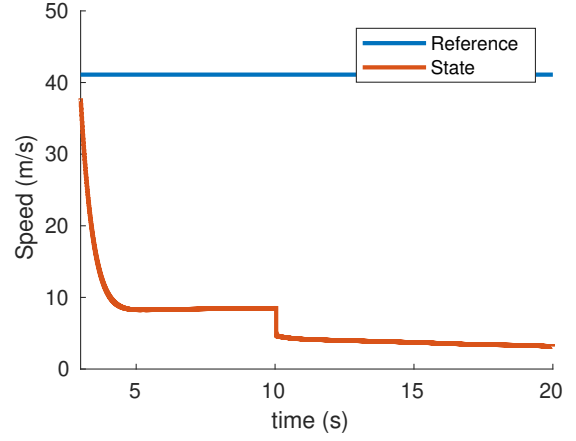
To test the controller, a fixed set-points of altitude and speed which are a bit deviated from the initial condition of the vehicle, as shown in Table 5.

Table 5. Controller Testing Condition

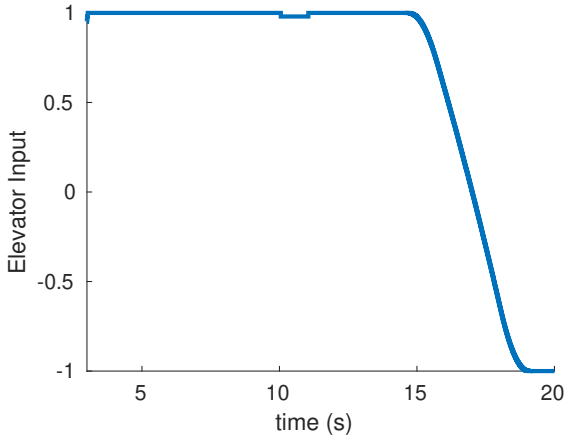
U_{ref} (m/s)	z_{ref} (m)	U_{init} (m/s)	z_{init} (m)	Θ_{init} (rad)
43.1	15	41.1	13	0.19



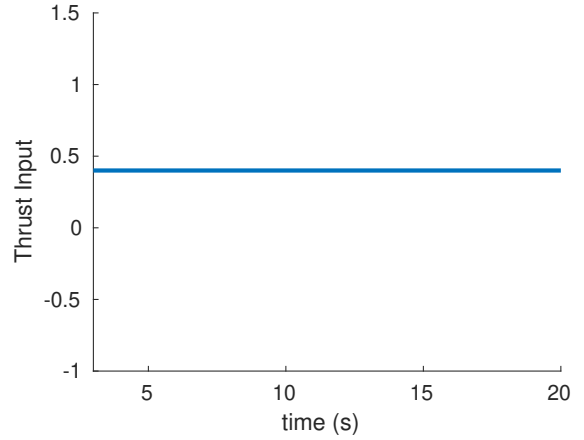
(a) Altitude controller



(b) Speed controller



(c) Elevator Input



(d) Thrust Input

Figure 8. First controller test result - Original Configuration

As shown in the plots in Figure 8, the vehicle suddenly lost a significant amount of speed, went below the stall speed, and plunged at around 10 seconds. However, Figure 8 (c) and (d) show that the speed and altitude controller were operational but capped at the allowed maximum effort. As the maximum thrust was given, the testing environment was set up again with the hypothesis that the LiftDragplugin would produce a thrust lower than the expected amount of thrust used in the state space analysis. The deeper analysis of the first testing result is handled in Section 4.

In the second test environment, therefore, the maximum thrust value was doubled by manually increasing the reference area of the blades in the pusher propeller.

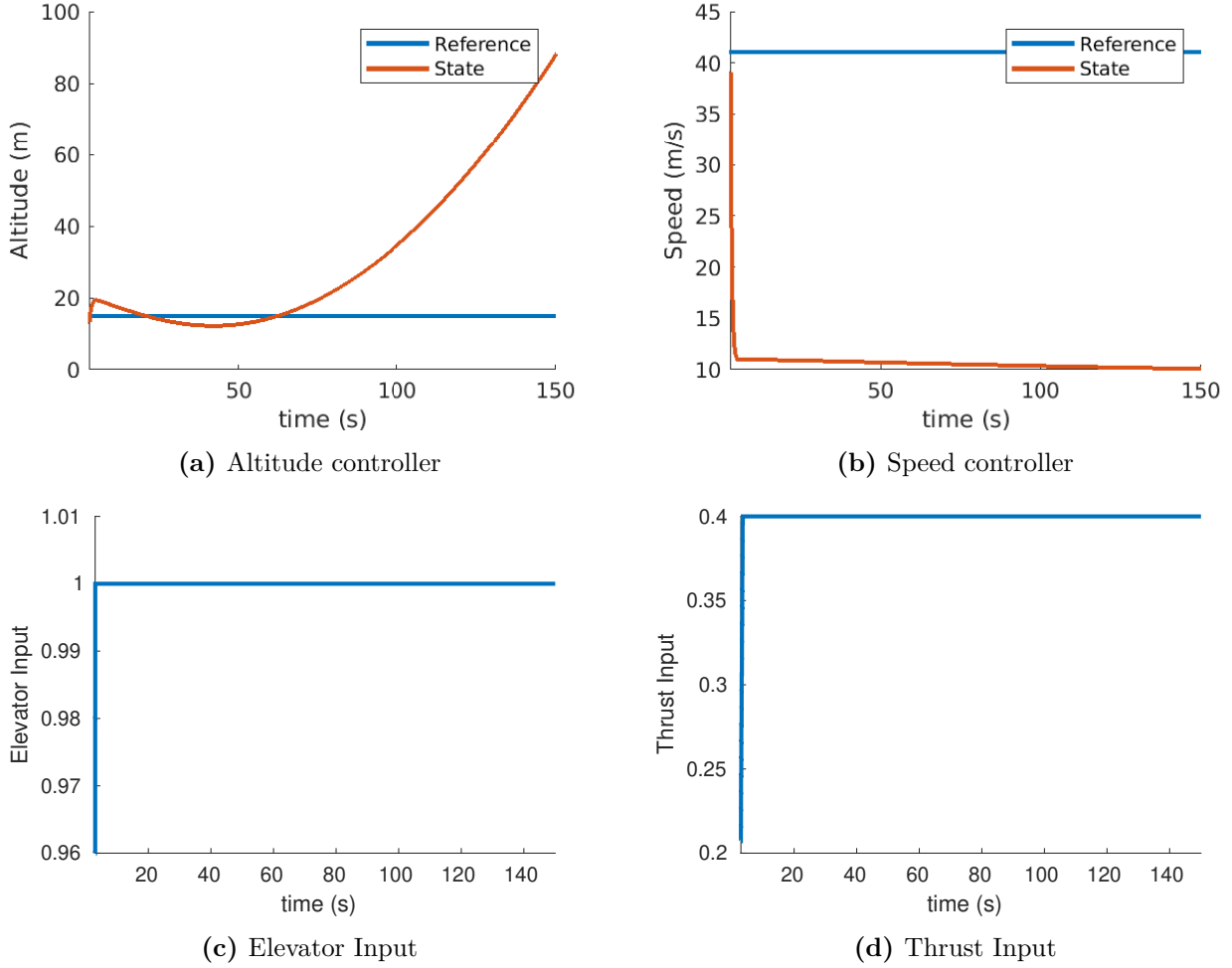
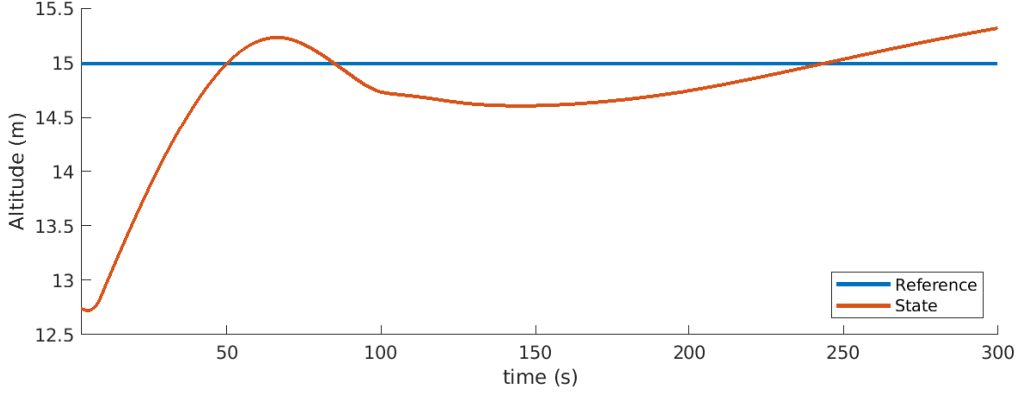


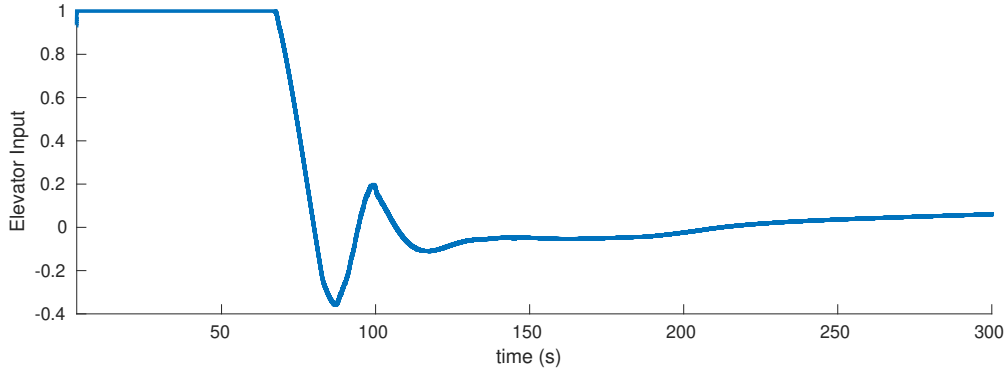
Figure 9. Second controller test result - Doubled thrust

Figure 9 shows that the vehicle did not plunge during the second controller test but still experienced a sudden drop in speed and failed to control the altitude and velocity. The result can be explained if the produced thrust generates a moment greater than that the horizontal tailplane can counteract. The final design of the vehicle locates the pusher propeller below the CG of the vehicle, contributing to the aircraft's nose-up attitude [30].

As both the original and thrust-increased configurations did not work, the third approach had to be proposed. The third simulation environment uses `setVelocity()` command to make the vehicle model track the reference velocity in real-time without a controller. As a result, the speed controller section of the longitudinal controller could not be used, but the altitude controller section could operate in an optimal condition. The testing result is represented in Figure 10. It is shown that the elevator controller operates appropriately with a certain degree of tracking ability. However, the altitude controller also showed a considerably long rise time and oscillatory altitude state.



(a) Altitude controller



(b) Elevator input

Figure 10. Third controller test result - Constant Velocity

4 Discussion and Conclusion

4.1 Limitations of Simulation Environment and Model

The simulation environment was expected to work with the first configuration, which fully utilises the modelled vehicle and the controller. However, as shown, the testing failed due to the sudden drop in velocity. During the testing process, the thrust value was assumed to be underestimated by Liftdragplugin. However, a more thorough analysis seemed to be required after the whole testing procedure.

Since no disturbance (e.g. gust) was introduced during the testing process, drag force is the only force that can decelerate the vehicle. Therefore, the magnitude of drag that should be generated based on the Liftdragplugin could be calculated based on Equation (1) but changing C_L to C_d . The result was then compared with the drag force required to generate the deceleration that happened in the first testing.

Table 6. First controller test: Drag force generated by Liftdragplugin and the required drag force

Lifting surface	AoA (rad)	Area (m^2)	C_d
Wing	0.216	4.24	0.0151
Horizontal Stabiliser		1.01	0.000648
Generated drag (N)		65.7	
Required drag (N)		5211	

Table 6 shows that the expected instantaneous drag force from Liftdragplugin is hugely less than the

drag that should be exerted on the vehicle to make the deceleration occur at the beginning of testing. It indicates the possibility of the discontinuity in the linear momentum of the vehicle.

At the same time, during the second test, it was shown that the amount of thrust could have a critical effect on the nose-up attitude of the vehicle. Therefore, the actual thrust generated by the pusher propeller with Liftdragplugin needed to be measured. To do so, a separate simulation environment with zero gravity was created. Then, the model was spawned up in the air with a pitch angle of 90 degrees. Finally, locking the roll and yaw motion of the vehicle allowed the measurement of the velocity and acceleration by measuring the linear velocity and acceleration of the vehicle in the z-direction.

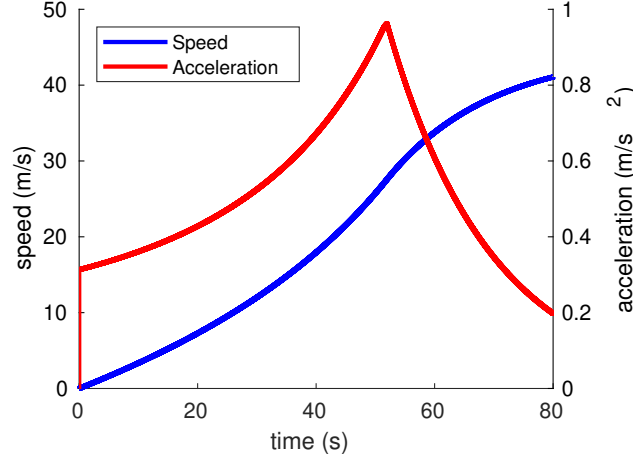


Figure 11. Velocity and acceleration generated by pusher propeller at maximum RPM

Table 3 and Figure 11 show that the pusher propeller shows the maximum acceleration slightly below the velocity at which the pusher propeller design was optimised. This is sensible as the thrust was measured at the maximum RPM of the motor, and the optimisation condition used 2000 RPM. However, as the propeller blade stalls, it cannot generate the maximum thrust of 320N, which the longitudinal controller expects [3]. This result indicates two possible errors. One is the systematic error when the propeller blade aerodynamics was re-modelled to use Liftdragplugin. Another possible error source is that the current design of the pusher propeller cannot generate the expected maximum thrust during the cruise as it was optimised for loitering [29].

Along with the sources of error stated above, other limitations do exist in the simulation environment. Vehicle modelling only uses the point mass value of each part of the vehicle to set the moment of inertia, assuming a constant density on every part. It can result in the difference in the inertial property of the actual vehicle and the virtual one used in this simulation as typically more than one material is used for each part of the aircraft [31].

The Liftdragplugin allows for the generation of aerodynamic force during the simulation, but the thin-aerofoil theory cannot be ignored. Thin-aerofoil theory often gives an acceptable estimation of lift and moment coefficient change with the variation AoA, but it is pointed out that it often gives an inaccurate estimation of drag [32]. At the same time, as the plugin only uses the velocity of the link where the centre of pressure is attached, it cannot reflect the interaction between the lifting surfaces of the vehicle, which especially has a significant impact on the aerodynamic forces on the empennage [33].

When giving command input to the model during the simulation, `setJointVelocity()` and `setJointPosition()` commands were used from Simulink and transported to Gazebo. In other words, the simulator instantaneously changed the motor's angular velocity and deflection angle based on the given command. The command value, therefore, had to be smoothed to prevent the highly discontinuous behaviour of the motor and control surface. Still, it could not perfectly simulate the low-level controllers that exist to control the motor speed and deflection angle [6].

4.2 Conclusion

As stated in the previous sections, several points should be addressed to improve the current simulation environment. To analyse the cause of the sudden drop in the vehicle speed, a detailed analysis of the Cosim plugin and the behaviour of ODE, a default physics engine used in Gazebo, will be required. Liftdragplugin relies on many assumptions and cannot reflect the complex behaviour of the flow. It especially introduced the difficulty in modelling the aerodynamics of the propeller. The aerodynamic forces generated by the plugin should be verified using Computational Fluid Dynamics and real-world experiments [34].

The low-level controller should be realised inside the simulation environment to replicate the real-world behaviour of the actuators more accurately. Gazebo provides a PID plugin which can be used for joint control, but the Cosim plugin does not support it [10]. Therefore, developing a custom plugin to apply the PID plugin would be required to improve the simulation environment.

Regarding the controller testing, the VTOL and transition controller tests must be done as soon as they are converted into an appropriate Simulink file. Then, the three controllers should be integrated and tested as a whole. After verifying that the combined controller shows an adequate performance, the simulation can be done in a more complex physical environment by introducing a realistic terrain and random wind. As a last phase, the simulation using both controller and navigation systems developed by the Systems and Propulsion team would be possible [35].

References

- [1] Hasan Rashid. Halo reach: Project coordination through systems engineering, risk analysis and conceptual design. GDP report 02020313, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [2] Adnan S Saeed, Ahmad Bani Younes, Shafiqul Islam, Jorge Dias, Lakmal Seneviratne, and Guowei Cai. A review on the platform design, dynamic modeling and control of hybrid uavs. In *2015 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 806–815. IEEE, 2015.
- [3] Thilak S Baikady. Halo reach: Longitudinal control design. GDP report 02016794, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [4] Harry Allen. Halo reach: Advanced uav quadrotor control: A comparison of non-linear mpc and rl agent methods. GDP report 02028116, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [5] Shun Tang. Halo reach: Vtol forward and backward transition controller development. GDP report 02023280, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [6] Karl Johan Åström. Control system design lecture notes for me 155a. *Department of Mechanical and Environmental Engineering University of California Santa Barbara*, 333, 2002.
- [7] NATO RESEARCH and TECHNOLOGY ORGANIZATION NEUILLY-SUR-SEINE (FRANCE). Agard flight test techniques series. volume 19. simulation in support of flight testing.(la simulation pour le soutien des essais en vol). 2000.
- [8] Panagiotis A Vitsas. Spin reproduction, control system testing and control room training using x-plane. 2013.
- [9] The MathWorks Inc. Guidance model, 2024. URL <https://uk.mathworks.com/help/uav/ref/guidancemodel.html>.
- [10] Open Source Robotics Foundation. Gazebo api reference. URL <https://osrf-distributions.s3.amazonaws.com/gazebo/api/dev/index.html>.
- [11] DARPA Outreach Team. Service academies swarm challenge pushes the boundaries of autonomous swarm capabilities, May 2017. URL <https://www.darpa.mil/news-events/2017-05-11>.
- [12] Open Source Robotics Foundation. Gazebo tutorial: Aerodynamics, 2014. URL <https://classic.gazebosim.org/tutorials?tut=aerodynamics>.
- [13] The MathWorks Inc. Gazebo co-simulation - matlab & simulink, 2024. URL https://uk.mathworks.com/help/robotics/gazebo-cosimulation.html?s_tid=CRUX_lftnav.
- [14] Gang Peng, Tin Lun Lam, Chunxu Hu, Yu Yao, Jintao Liu, and Fan Yang. *Robot System Modeling*, pages 139–189. Springer Nature Singapore, Singapore, 2023. ISBN 978-981-99-1814-0. doi: 10.1007/978-981-99-1814-0_3. URL https://doi.org/10.1007/978-981-99-1814-0_3.
- [15] ArduPilot development team. Alti transition quadplane. https://github.com/ArduPilot/SITL_Models/blob/master/Gazebo/docs/AltiTransition.md, 2024.
- [16] Alti transition. URL <https://www.altiuas.com/transition>.
- [17] Open Source Robotics Foundation. Sdformat, 2020. URL <http://sdformat.org/>.
- [18] Shivani Chauhan. Halo reach: Fuselage and boom structural design. GDP report 02096006, Dept. of Aeronautical Engineering, Imperial College London, 2024.

- [19] Louis Nelissen. Development of gazebo-based high-fidelity simulation environment for morphing-wing uavs. Master’s thesis, Robotics and Mechatronics (RaM) group, University of Twente, 2021.
- [20] LC Woods. Generalized aerofoil theory. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 238(1214):358–388, 1957.
- [21] Sriram K Rallabhandi, EroI Cagatay, and Dimitri N Mavris. An improved procedure for prediction of drag polars of a joined wing concept using physics-based response surface methodology. *SAE transactions*, pages 653–662, 2001.
- [22] Michael S Selig. Uiuc airfoil data site. 1996.
- [23] Matthew M Homa. Halo reach: Wing design and computational propeller analysis. GDP report 02038403, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [24] Naboth Dirirsa. Halo reach: Design and optimisation of winglets & aerodynamic tailplane enhancements. GDP report 02026211, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [25] Mohammad H Sadraey. *Aircraft design: A systems engineering approach*. John Wiley & Sons, 2012.
- [26] E Eugene Larrabee. Design of propellers for motorsoarers. *NASA. Langley Res. Center The Sci. and Technol. of Low Speed and Motorless Flight, Pt. 1*, 1979.
- [27] Shahriar H Alam. Halo reach: Design and optimisation of vtol rotor with computational analysis. GDP report 02037395, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [28] John Anderson. *EBOOK: Fundamentals of Aerodynamics (SI units)*. McGraw hill, 2011.
- [29] Parasdeep Singh. Halo reach: Design and optimisation of pusher propeller. GDP report 02099678, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [30] Tharm Chursuk. Halo reach: Optimising weight and balance through comprehensive powerplant sizing for vtol fixed-wing uav. GDP report 02048996, Dept. of Aeronautical Engineering, Imperial College London, 2024.
- [31] Adrian P Mouritz. *Introduction to aerospace materials*. Elsevier, 2012. doi: <https://doi.org/10.1533/9780857095152.173>.
- [32] N. A. Ahmed. Demonstration of the effectiveness and limitations of thin airfoil theory in the aerodynamic study of airfoil characteristics. *International Journal of Mechanical Engineering Education*, 32(4):271–282, 2004. doi: 10.7227/IJMEE.32.4.1. URL <https://doi.org/10.7227/IJMEE.32.4.1>.
- [33] Errikos L. Intro to aerospace, 2021- 2022. URL https://bb.imperial.ac.uk/ultra/courses/_18483_1/cl/outline.
- [34] Erwhin Irmawan, Agus Harjoko, and Andi Dharmawan. Model, control, and realistic visual 3d simulation of vtol fixed-wing transition flight considering ground effect. *Drones*, 7(5):330, 2023.
- [35] Divij Wankhade. Halo reach: Navigation, communications and data relay system. GDP report 02027906, Dept. of Aeronautical Engineering, Imperial College London, 2024.

Appendix

A Drag Polar and Momentum Curve of the Used Aerofoils

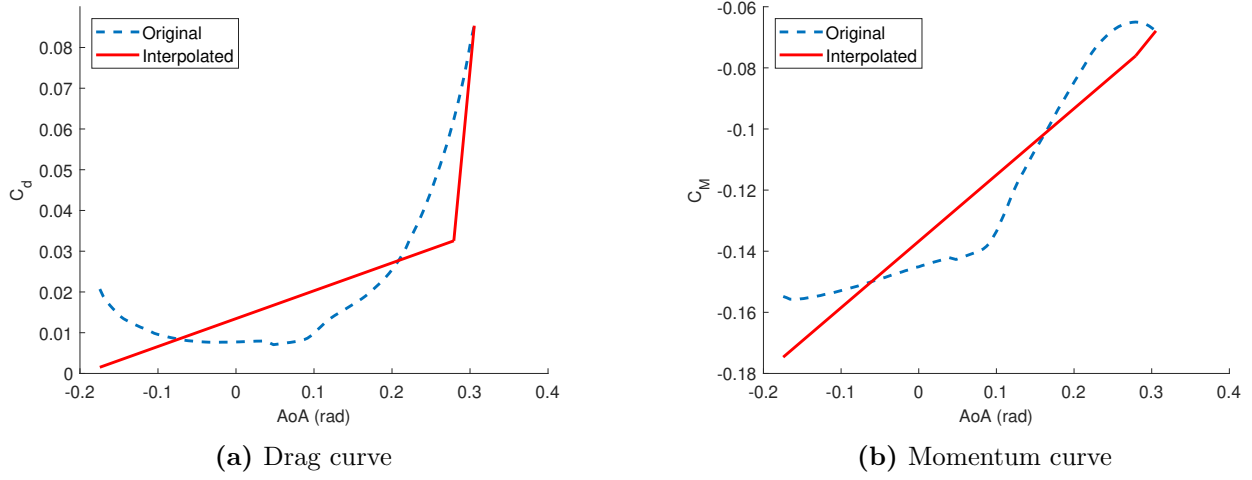


Figure 12. NACA-6409

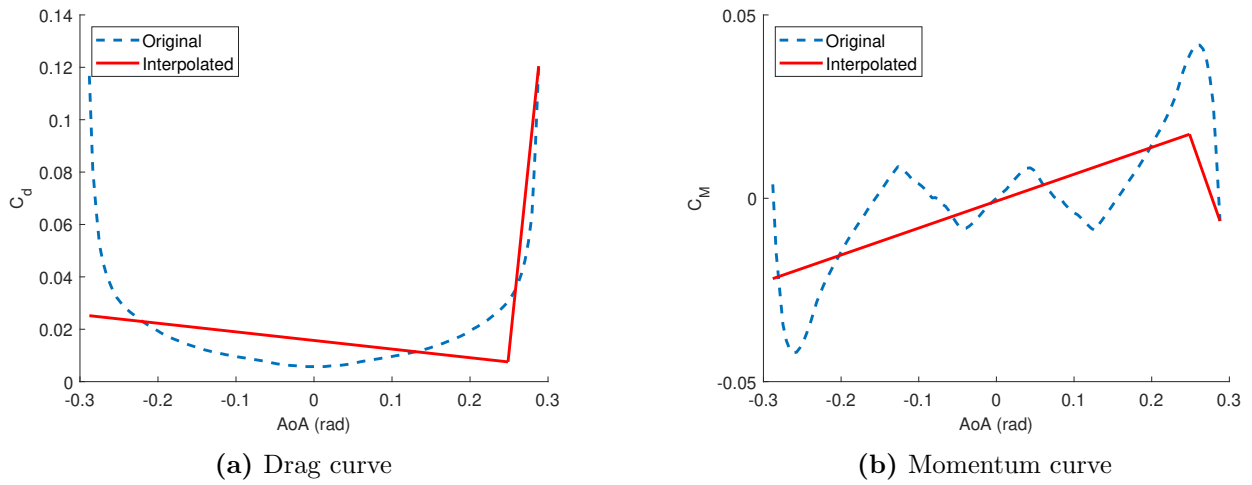
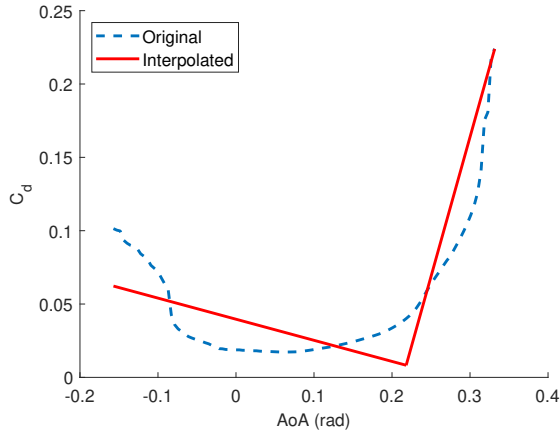
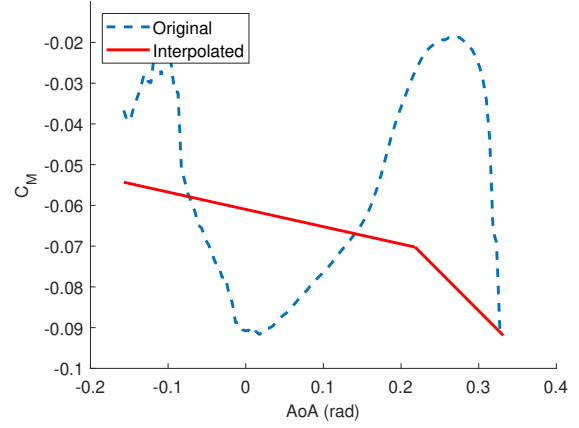


Figure 13. NACA-0009

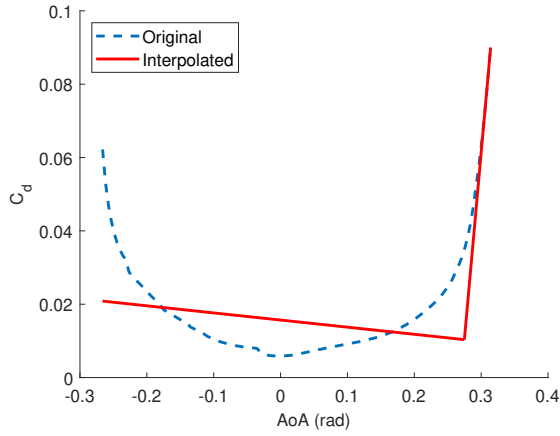


(a) Drag curve

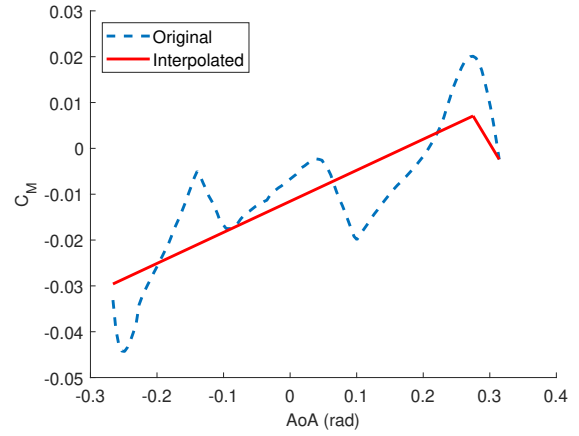


(b) Momentum curve

Figure 14. Clark-Y

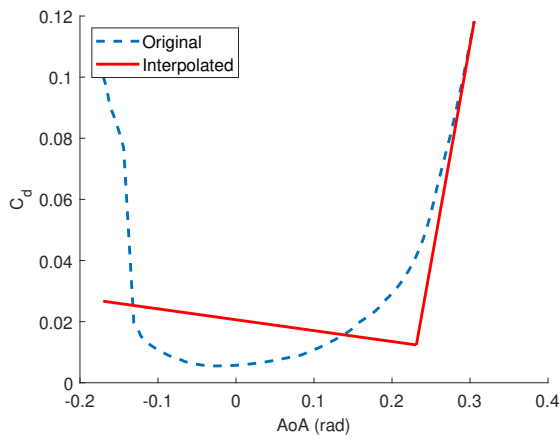


(a) Drag curve

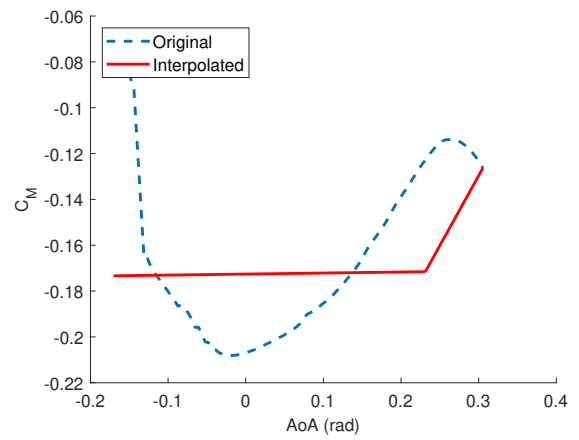


(b) Momentum curve

Figure 15. NACA-23012



(a) Drag curve



(b) Momentum curve

Figure 16. E216

B The code for Gazebo Simulation - Model and Liftdragplugin

B.1 Model Code

```
1 <?xml version="1.0"?>
2 <sdf version="1.7">
3   <world name="default">
4     <include>
5       <uri>model://ground_plane</uri>
6     </include>
7
8     <include>
9       <uri>model://sun</uri>
10    </include>
11
12    <model name="Fixed_Wing_VTOL">
13      <pose>0 0 1 0 0 0</pose> <!--The initial Position of the model-->
14
15      <!--Base_Link: Wing, Fuselage, Undercarriage, empennage-->
16      <!-- Edited 9th June 2024 -->
17      <link name="base_link">
18        <pose>0 0 0.109952863 0 0 0</pose>
19        <inertial>
20          <pose>0 0 0 0 0 0</pose>
21          <mass>93.235</mass>
22          <inertia>
23            <ixx>11.03256327</ixx>
24            <ixy>0</ixy>
25            <ixz>6.003311401</ixz>
26            <iyy>58.30874301</iyy>
27            <iyz>0</iyz>
28            <izz>63.11367974</izz>
29          </inertia>
30        </inertial>
31
32        <!-- Collision Information. DAE File of Vehicle Model goes in -->
33        <!-- NOT CRUCIAL -->
34        <collision name="fuselage_collision">
35          <pose relative_to="base_link">-0.584031951 0 -0.109952863 0 0 0</pose>
36          <geometry>
37            <mesh>
38              <uri>file://Base.dae</uri>
39            </mesh>
40          </geometry>
41        </collision>
42
43        <!-- Visual Information -->
44        <!--NOT CRUCIAL-->
45        <visual name="base_link_visual">
46          <pose relative_to="base_link">-0.584031951 0 -0.109952863 0 0 0</pose>
47          <geometry>
48            <mesh>
49              <uri>file://Base.dae</uri>
50            </mesh>
51          </geometry>
52          <material>
53            <script>
54              <name>Gazebo/Grey</name>
55              <uri>__default__</uri>
56            </script>
57          </material>
58        </visual>
59      </link>
60    </model>
  </world>
</sdf>
```

```

61
62 <!--
63 <link name="payload">
64   <pose>0.775 0 0 0 0 0</pose>
65   <inertial>
66     <pose>0.775 0 0 0 0 0</pose>
67     <mass>50</mass>
68     <inertia>
69       <ixx>11.46563192</ixx>
70       <ixy>0</ixy>
71       <ixz>14.26668265</ixz>
72       <iyy>79.75055313</iyy>
73       <iyz>0</iyz>
74       <izz>181.0044921</izz>
75     </inertia>
76   </inertial>
77
78   <collision name="payload_collision">
79     <geometry>
80       <box>
81         <size>0.369 0.369 0.369</size>
82       </box>
83     </geometry>
84   </collision>
85
86   <visual name="payload_visual">
87     <geometry>
88       <box>
89         <size>0.369 0.369 0.369</size>
90       </box>
91     </geometry>
92     <material>
93       <script>
94         <name>Gazebo/Grey</name>
95         <uri>__default__</uri>
96       </script>
97     </material>
98   </visual>
99
100 </link>
101 -->
102
103 <!-- Links of Motors (VTOL) -->
104 <!-- Edited 9th June 2024 -->
105 <link name="motor_1">
106   <pose>1.115968049 -1.25 0.275 0 0 0</pose>
107   <inertial>
108     <pose>0 0 0 0 0 0</pose>
109     <mass>3.1</mass>
110     <inertia>
111       <ixx>0.004644979</ixx>
112       <ixy>0</ixy>
113       <ixz>0</ixz>
114       <iyy>0.004644979</iyy>
115       <iyz>0</iyz>
116       <izz>0.00871875</izz>
117     </inertia>
118   </inertial>
119   <collision name="motor_1_collision">
120     <pose>-0.25 0 0.01 0 0 0</pose>
121     <geometry>
122       <cylinder>
123         <length>0.005</length>
124         <radius>0.2286</radius> <!-- Propeller Blade Radius goes in -->
125       </cylinder>

```

```

126     </geometry>
127 </collision>
128 <visual name="motor_1_visual">
129   <pose>-0.25 0 0.01 0 0 0</pose>
130   <geometry>
131     <mesh>
132       <scale>2.5 2.5 2.5</scale>
133       <uri>file://prop_cw.dae</uri>
134     </mesh>
135   </geometry>
136   <material>
137     <script>
138       <name>Gazebo/Blue</name>
139       <uri>__default__</uri>
140     </script>
141   </material>
142 </visual>
143 </link>
144 <joint name="motor_1_joint" type="revolute">
145   <child>motor_1</child>
146   <parent>base_link</parent>
147   <axis>
148     <xyz>0 0 1</xyz>
149     <limit>
150       <lower>-1e+16</lower> <!-- Limit Angular Velocity (rad/s) -->
151       <upper>1e+16</upper>
152     </limit>
153     <dynamics>
154       <damping>0.002</damping>
155     </dynamics>
156     <use_parent_model_frame>1</use_parent_model_frame>
157   </axis>
158 </joint>
159
160 <link name="motor_2">
161   <pose>-1.004031951 1.25 0.275 0 0 0</pose>
162   <inertial>
163     <pose>0 0 0 0 0 0</pose>
164     <mass>3.1</mass>
165     <inertia>
166       <ixx>0.004644979</ixx>
167       <ixy>0</ixy>
168       <ixz>0</ixz>
169       <iyy>0.004644979</iyy>
170       <iyz>0</iyz>
171       <izz>0.00871875</izz>
172     </inertia>
173   </inertial>
174   <collision name="motor_2_collision">
175     <pose>-0.3 0 0.01 0 0 0</pose>
176     <geometry>
177       <cylinder>
178         <length>0.005</length>
179         <radius>0.2286</radius> <!-- Propeller Blade Radius goes in -->
180       </cylinder>
181     </geometry>
182   </collision>
183   <visual name="motor_2_visual">
184     <pose>-0.3 0 0.01 0 0 0</pose>
185     <geometry>
186       <mesh>
187         <scale>2.5 2.5 2.5</scale>
188         <uri>file://prop_cw.dae</uri>
189       </mesh>
190     </geometry>

```

```

191     <material>
192       <script>
193         <name>Gazebo/Blue</name>
194         <uri>__default__</uri>
195       </script>
196     </material>
197   </visual>
198 </link>
199 <joint name="motor_2_joint" type="revolute">
200   <child>motor_2</child>
201   <parent>base_link</parent>
202   <axis>
203     <xyz>0 0 1</xyz>
204     <limit>
205       <lower>-1e+16</lower>
206       <upper>1e+16</upper>
207     </limit>
208     <dynamics>
209       <damping>0.002</damping>
210     </dynamics>
211     <use_parent_model_frame>1</use_parent_model_frame>
212   </axis>
213 </joint>
214
215 <link name="motor_3">
216   <pose>1.115968049 1.25 0.275 0 0 0</pose>
217   <inertial>
218     <pose>0 0 0 0 0 0</pose>
219     <mass>3.1</mass>
220     <inertia>
221       <ixx>0.004644979</ixx>
222       <ixy>0</ixy>
223       <ixz>0</ixz>
224       <iyy>0.004644979</iyy>
225       <iyz>0</iyz>
226       <izz>0.00871875</izz>
227     </inertia>
228   </inertial>
229   <collision name="motor_3_collision">
230     <pose>-0.25 0 0.01 0 0 0</pose>
231     <geometry>
232       <cylinder>
233         <length>0.005</length>
234         <radius>0.2286</radius> <!-- Propeller Blade Radius goes in -->
235       </cylinder>
236     </geometry>
237   </collision>
238   <visual name="motor_3_visual">
239     <pose>-0.25 0 0.01 0 0 0</pose>
240     <geometry>
241       <mesh>
242         <scale>2.5 2.5 2.5</scale>
243         <uri>file://prop_ccw.dae</uri>
244       </mesh>
245     </geometry>
246     <material>
247       <script>
248         <name>Gazebo/Blue</name>
249         <uri>__default__</uri>
250       </script>
251     </material>
252   </visual>
253 </link>
254 <joint name="motor_3_joint" type="revolute">
255   <child>motor_3</child>

```

```

256     <parent>base_link</parent>
257     <axis>
258         <xyz>0 0 1</xyz>
259         <limit>
260             <lower>-1e+16</lower>
261             <upper>1e+16</upper>
262         </limit>
263         <dynamics>
264             <damping>0.002</damping>
265         </dynamics>
266         <use_parent_model_frame>1</use_parent_model_frame>
267     </axis>
268 </joint>
269
270 <link name="motor_4">
271     <pose>-1.004031951 -1.25 0.275 0 0 0</pose>
272     <inertial>
273         <pose>0 0 0 0 0 0</pose>
274         <mass>3.1</mass>
275         <inertia>
276             <ixx>0.004644979</ixx>
277             <ixy>0</ixy>
278             <ixz>0</ixz>
279             <iyy>0.004644979</iyy>
280             <iyz>0</iyz>
281             <izz>0.00871875</izz>
282         </inertia>
283     </inertial>
284     <collision name="motor_4_collision">
285         <pose>-0.3 0 0.01 0 0 0</pose>
286         <geometry>
287             <cylinder>
288                 <length>0.005</length>
289                 <radius>0.2286</radius>
290             </cylinder>
291         </geometry>
292     </collision>
293     <visual name="motor_4_visual">
294         <pose>-0.3 0 0.01 0 0 0</pose>
295         <geometry>
296             <mesh>
297                 <scale>2.5 2.5 2.5</scale>
298                 <uri>file://prop_ccw.dae</uri>
299             </mesh>
300         </geometry>
301         <material>
302             <script>
303                 <name>Gazebo/Blue</name>
304                 <uri>__default__</uri>
305             </script>
306         </material>
307     </visual>
308 </link>
309 <joint name="motor_4_joint" type="revolute">
310     <child>motor_4</child>
311     <parent>base_link</parent>
312     <axis>
313         <xyz>0 0 1</xyz>
314         <limit>
315             <lower>-1e+16</lower>
316             <upper>1e+16</upper>
317         </limit>
318         <dynamics>
319             <damping>0.002</damping>
320         </dynamics>

```

```

321     <use_parent_model_frame>1</use_parent_model_frame>
322 </axis>
323 </joint>
324
325 <!-- Links of Motor (Pusher) -->
326 <!-- Edited 9th June 2024 -->
327 <link name="motor_f">
328   <pose>-0.584031951 0 0 0 -1.570796327 0</pose>
329   <inertial>
330     <mass>0.56</mass>
331     <inertia>
332       <ixx>9.75e-06</ixx>
333       <ixy>0</ixy>
334       <ixz>0</ixz>
335       <iyy>0.000166704</iyy>
336       <iyz>0</iyz>
337       <izz>0.000167604</izz>
338     </inertia>
339   </inertial>
340   <collision name="motor_f_collision">
341     <pose>0 0 0.2 0 0 0</pose>
342     <geometry>
343       <cylinder>
344         <length>0.03</length>
345         <radius>0.25</radius>
346       </cylinder>
347     </geometry>
348   </collision>
349   <visual name="motor_f_visual">
350     <pose>0 0 0.1 0 0 0</pose>
351     <geometry>
352       <mesh>
353         <scale>2.5 2.5 2.5</scale>
354         <uri>file://prop_ccw.dae</uri>
355       </mesh>
356     </geometry>
357     <material>
358       <script>
359         <name>Gazebo/Blue</name>
360         <uri>__default__</uri>
361       </script>
362     </material>
363   </visual>
364 </link>
365 <joint name="motor_f_joint" type="revolute">
366   <child>motor_f</child>
367   <parent>base_link</parent>
368   <axis>
369     <xyz>0 0 1</xyz>
370   <limit>
371     <lower>-1e+16</lower>
372     <upper>1e+16</upper>
373   </limit>
374   <dynamics>
375     <damping>0.002</damping>
376   </dynamics>
377   <use_parent_model_frame>1</use_parent_model_frame>
378 </axis>
379 </joint>
380
381
382 <!-- Left/Right Aileron Links -->
383 <!-- Edited on 11th June -->
384 <link name="left_aileron_link">
385   <pose relative_to="left_aileron_joint">0 0 0 -0.087266463 0 -0.09</pose>

```

```

386     <inertial>
387       <mass>0.1</mass>
388       <inertia>
389         <ixx>0.000102319</ixx>
390         <ixy>0</ixy>
391         <iyy>0.00334417</iyy>
392         <ixz>0</ixz>
393         <iyz>0</iyz>
394         <izz>0.003446072</izz>
395       </inertia>
396     </inertial>
397
398     <visual name="left_aileron_visual">
399       <geometry>
400         <mesh>
401           <scale>1.7 1.7 1.7</scale>
402           <uri>file://aileron_left.dae</uri>
403         </mesh>
404       </geometry>
405       <material>
406         <script>
407           <name>Gazebo/Blue</name>
408           <uri>__default__</uri>
409         </script>
410       </material>
411     </visual>
412 </link>
413
414 <!-- Joint range -30/+30 deg. -->
415 <joint name="left_aileron_joint" type="revolute">
416   <parent>base_link</parent>
417   <child>left_aileron_link</child>
418   <pose relative_to="base_link">0.8 1.15 0.244 0.087266463 0 0.034906585</pose>
419   <axis>
420     <xyz>0 1 0</xyz>
421     <limit>
422       <lower>-0.53</lower>
423       <upper>0.53</upper>
424     </limit>
425     <dynamics>
426       <damping>0.1</damping>
427     </dynamics>
428   </axis>
429 </joint>
430
431 <link name="right_aileron_link">
432   <pose relative_to="right_aileron_joint">0 0 0 0.087266463 0 0.09</pose>
433   <inertial>
434     <mass>0.1</mass>
435     <inertia>
436       <ixx>0.000102319</ixx>
437       <ixy>0</ixy>
438       <iyy>0.00334417</iyy>
439       <ixz>0</ixz>
440       <iyz>0</iyz>
441       <izz>0.003446072</izz>
442     </inertia>
443   </inertial>
444
445   <visual name="right_aileron_visual">
446     <geometry>
447       <mesh>
448         <scale>1.7 1.7 1.7</scale>
449         <uri>file://aileron_right.dae</uri>
450       </mesh>

```

```

451     </geometry>
452     <material>
453       <script>
454         <name>Gazebo/Blue</name>
455         <uri>__default__</uri>
456       </script>
457     </material>
458   </visual>
459 </link>
460
461 <!-- Joint range -30/+30 deg. -->
462 <joint name="right_aileron_joint" type="revolute">
463   <parent>base_link</parent>
464   <child>right_aileron_link</child>
465   <pose relative_to="base_link">0.8 -1.15 0.244 -0.087266463 0 -0.034906585</pose>
466   <axis>
467     <xyz>0 1 0</xyz>
468     <limit>
469       <lower>-0.53</lower>
470       <upper>0.53</upper>
471     </limit>
472     <dynamics>
473       <damping>0.1</damping>
474     </dynamics>
475   </axis>
476 </joint>
477
478 <!-- Elevator Link -->
479 <!-- Edited on 11th June -->
480 <link name="elevator_link">
481   <pose relative_to="elevator_joint">0 0 0 0 0 0</pose>
482   <inertial>
483     <mass>0.1</mass>
484     <inertia>
485       <ixx>0.000102319</ixx>
486       <ixy>0</ixy>
487       <iyy>0.00334417</iyy>
488       <ixz>0</ixz>
489       <iyz>0</iyz>
490       <izz>0.003446072</izz>
491     </inertia>
492   </inertial>
493   <collision name="elevator_collision">
494     <pose>-0.03 0 0 0 0 0</pose>
495     <geometry>
496       <box>
497         <size>0.12 1.0 0.01</size>
498       </box>
499     </geometry>
500   </collision>
501   <visual name="elevator_visual">
502     <geometry>
503       <mesh>
504         <scale>1.7 1.7 1.7</scale>
505         <uri>file:///elevator.dae</uri>
506       </mesh>
507     </geometry>
508     <material>
509       <script>
510         <name>Gazebo/Blue</name>
511         <uri>__default__</uri>
512       </script>
513     </material>
514   </visual>
515 </link>

```

```

516
517 <!-- Joint range -30/+30 deg. -->
518 <joint name="elevator_joint" type="revolute">
519   <parent>base_link</parent>
520   <child>elevator_link</child>
521   <pose relative_to="base_link">-2.8 0 1.15 0 0 0</pose>
522   <axis>
523     <xyz>0 1 0</xyz>
524     <limit>
525       <lower>-0.53</lower>
526       <upper>0.53</upper>
527     </limit>
528     <dynamics>
529       <damping>0.1</damping>
530     </dynamics>
531   </axis>
532 </joint>
533
534 <!-- Left/Right Rudder Links -->
535 <!-- Edited on 11th June -->
536
537 <!-- Left Rudder Link -->
538 <link name="left_rudder_link">
539   <pose relative_to="left_rudder_joint">0 0 0 0 0 0</pose>
540   <inertial>
541     <mass>0.1</mass>
542     <inertia>
543       <ixx>0.000102319</ixx>
544       <ixy>0</ixy>
545       <iyy>0.00334417</iyy>
546       <ixz>0</ixz>
547       <iyz>0</iyz>
548       <izz>0.003446072</izz>
549     </inertia>
550   </inertial>
551   <visual name="left_rudder_visual">
552     <geometry>
553       <mesh>
554         <uri>file://aileron_left.dae</uri>
555       </mesh>
556     </geometry>
557     <material>
558       <script>
559         <name>Gazebo/Blue</name>
560         <uri>__default__</uri>
561       </script>
562     </material>
563   </visual>
564 </link>
565
566 <!-- Joint range -30/+30 deg. -->
567 <joint name="left_rudder_joint" type="revolute">
568   <parent>base_link</parent>
569   <child>left_rudder_link</child>
570   <pose relative_to="base_link">-2.75 1.25 0.8 1.5 0 -0.2</pose>
571   <axis>
572     <xyz>0 1 0</xyz>
573     <limit>
574       <lower>-0.62</lower>
575       <upper>0.62</upper>
576     </limit>
577     <dynamics>
578       <damping>0.1</damping>
579     </dynamics>
580   </axis>

```

```

581 </joint>
582
583 <!-- right Rudder Link-->
584 <link name="right_rudder_link">
585   <pose relative_to="right_rudder_joint">0 0 0 0 0 0</pose>
586   <inertial>
587     <mass>0.1</mass>
588     <inertia>
589       <ixx>0.000102319</ixx>
590       <ixy>0</ixy>
591       <iyy>0.00334417</iyy>
592       <ixz>0</ixz>
593       <iyz>0</iyz>
594       <izz>0.003446072</izz>
595     </inertia>
596   </inertial>
597
598   <visual name="right_rudder_visual">
599     <geometry>
600       <mesh>
601         <uri>file://aileron_right.dae</uri>
602       </mesh>
603     </geometry>
604     <material>
605       <script>
606         <name>Gazebo/Blue</name>
607         <uri>__default__</uri>
608       </script>
609     </material>
610   </visual>
611 </link>
612
613 <!-- Joint range -30/+30 deg. -->
614 <joint name="right_rudder_joint" type="revolute">
615   <parent>base_link</parent>
616   <child>right_rudder_link</child>
617   <pose relative_to="base_link">-2.75 -1.25 0.8 1.5 0 -0.2</pose>
618   <axis>
619     <xyz>0 1 0</xyz>
620     <limit>
621       <lower>-0.62</lower>
622       <upper>0.62</upper>
623     </limit>
624     <dynamics>
625       <damping>0.1</damping>
626     </dynamics>
627   </axis>
628 </joint>
629
630 <!-- sensor (IMU) Link -->
631 <!-- No need to be edited-->
632 <link name="imu_link">
633   <inertial>
634     <pose>0 0 0 0 0 0</pose>
635     <mass>0.15</mass>
636     <inertia>
637       <ixx>0.00002</ixx>
638       <ixy>0</ixy>
639       <ixz>0</ixz>
640       <iyy>0.00002</iyy>
641       <iyz>0</iyz>
642       <izz>0.00002</izz>
643     </inertia>
644   </inertial>
645   <sensor name="imu_sensor" type="imu">

```

```

646     <pose>0 0 0 0 0 0</pose>
647     <always_on>1</always_on>
648     <update_rate>1000.0</update_rate>
649 </sensor>
650 </link>
651 <joint name="imu_joint" type="revolute">
652   <child>imu_link</child>
653   <parent>base_link</parent>
654   <axis>
655     <xyz>0 0 1</xyz>
656     <limit>
657       <lower>0</lower>
658       <upper>0</upper>
659       <effort>0</effort>
660       <velocity>0</velocity>
661     </limit>
662     <dynamics>
663       <damping>1.0</damping>
664     </dynamics>
665     <use_parent_model_frame>1</use_parent_model_frame>
666   </axis>
667 </joint>
668 </model>
669
670
671 <gravity>0 0 -9.81</gravity>
672
673 <!-- Plugin for connecting Gazebo and Simulink Controller -->
674 <plugin name="GazeboPlugin" filename="libGazeboCoSimPlugin.so"><portNumber>14581</portNumber></
    plugin>
675
676 </world>
677 </sdf>
678
679 <!-- End of the code -->

```

B.2 Liftdragplugin Code

```

1 <!-- motor_1 (ccw) lift-drag -->
2 <!-- Try Using both Clark y and NACA 23012-->
3 <plugin filename="libLiftDragPlugin.so"
4   name="Motor1_1_LiftDrag">
5   <a0>0.2337</a0>
6   <alpha_stall>0.2437</alpha_stall>
7   <cla>5.8839</cla>
8   <cda>0.08739</cda>
9   <cma>-0.0071</cma>
10  <cla_stall>-3.4342</cla_stall>
11  <cda_stall>1.3866</cda_stall>
12  <cma_stall>0.49</cma_stall>
13  <area>0.041</area>
14  <air_density>1.2041</air_density>
15  <cp>0.3 0 0</cp>
16  <forward>0 1 0</forward>
17  <upward>0 0 1</upward>
18  <link_name>motor_1</link_name>
19 </plugin>
20 <plugin filename="libLiftDragPlugin.so"
21   name="Motor1_2_LiftDrag">
22   <a0>0.2337</a0>
23   <alpha_stall>0.2437</alpha_stall>
24   <cla>5.8839</cla>

```

```

25     <cda>0.08739</cda>
26     <cma>-0.0071</cma>
27     <cla_stall>-3.4342</cla_stall>
28     <cda_stall>1.3866</cda_stall>
29     <cma_stall>0.49</cma_stall>
30     <area>0.041</area>
31     <air_density>1.2041</air_density>
32     <cp>-0.3 0 0</cp>
33     <forward>0 -1 0</forward>
34     <upward>0 0 1</upward>
35     <link_name>motor_1</link_name>
36 </plugin>
37 <!-- motor_2 (ccw) lift-drag -->
38 <plugin filename="libLiftDragPlugin.so"
39     name="Motor2_1_LiftDrag">
40     <a0>0.2337</a0>
41     <alpha_stall>0.2437</alpha_stall>
42     <cla>5.8839</cla>
43     <cda>0.08739</cda>
44     <cma>-0.0071</cma>
45     <cla_stall>-3.4342</cla_stall>
46     <cda_stall>1.3866</cda_stall>
47     <cma_stall>0.49</cma_stall>
48     <area>0.041</area>
49     <air_density>1.2041</air_density>
50     <cp>0.3 0 0</cp>
51     <forward>0 1 0</forward>
52     <upward>0 0 1</upward>
53     <link_name>motor_2</link_name>
54 </plugin>
55 <plugin filename="libLiftDragPlugin.so"
56     name="Motor2_2_LiftDrag">
57     <a0>0.2337</a0>
58     <alpha_stall>0.2437</alpha_stall>
59     <cla>5.8839</cla>
60     <cda>0.08739</cda>
61     <cma>-0.0071</cma>
62     <cla_stall>-3.4342</cla_stall>
63     <cda_stall>1.3866</cda_stall>
64     <cma_stall>0.49</cma_stall>
65     <area>0.041</area>
66     <air_density>1.2041</air_density>
67     <cp>-0.3 0 0</cp>
68     <forward>0 -1 0</forward>
69     <upward>0 0 1</upward>
70     <link_name>motor_2</link_name>
71 </plugin>
72 <!-- motor_3 (cw) lift-drag -->
73 <plugin filename="libLiftDragPlugin.so"
74     name="Motor3_1_LiftDrag">
75     <a0>0.2337</a0>
76     <alpha_stall>0.2437</alpha_stall>
77     <cla>5.8839</cla>
78     <cda>0.08739</cda>
79     <cma>-0.0071</cma>
80     <cla_stall>-3.4342</cla_stall>
81     <cda_stall>1.3866</cda_stall>
82     <cma_stall>0.49</cma_stall>
83     <area>0.041</area>
84     <air_density>1.2041</air_density>
85     <cp>0.3 0 0</cp>
86     <forward>0 -1 0</forward>
87     <upward>0 0 1</upward>
88     <link_name>motor_3</link_name>
89 </plugin>

```

```

90 <plugin filename="libLiftDragPlugin.so"
91     name="Motor3_2_LiftDrag">
92     <a0>0.2337</a0>
93     <alpha_stall>0.2437</alpha_stall>
94     <cla>5.8839</cla>
95     <cda>0.08739</cda>
96     <cma>-0.0071</cma>
97     <cla_stall>-3.4342</cla_stall>
98     <cda_stall>1.3866</cda_stall>
99     <cma_stall>0.49</cma_stall>
100    <area>0.041</area>
101    <air_density>1.2041</air_density>
102    <cp>-0.3 0 0</cp>
103    <forward>0 1 0</forward>
104    <upward>0 0 1</upward>
105    <link_name>motor_3</link_name>
106 </plugin>
107 <!-- motor_4 (cw) lift-drag -->
108 <plugin filename="libLiftDragPlugin.so"
109     name="Motor4_1_LiftDrag">
110     <a0>0.2337</a0>
111     <alpha_stall>0.2437</alpha_stall>
112     <cla>5.8839</cla>
113     <cda>0.08739</cda>
114     <cma>-0.0071</cma>
115     <cla_stall>-3.4342</cla_stall>
116     <cda_stall>1.3866</cda_stall>
117     <cma_stall>0.49</cma_stall>
118     <area>0.041</area>
119     <air_density>1.2041</air_density>
120     <cp>0.3 0 0</cp>
121     <forward>0 -1 0</forward>
122     <upward>0 0 1</upward>
123     <link_name>motor_4</link_name>
124 </plugin>
125 <plugin filename="libLiftDragPlugin.so"
126     name="Motor4_2_LiftDrag">
127     <a0>0.2337</a0>
128     <alpha_stall>0.2437</alpha_stall>
129     <cla>5.8839</cla>
130     <cda>0.08739</cda>
131     <cma>-0.0071</cma>
132     <cla_stall>-3.4342</cla_stall>
133     <cda_stall>1.3866</cda_stall>
134     <cma_stall>0.49</cma_stall>
135     <area>0.041</area>
136     <air_density>1.2041</air_density>
137     <cp>-0.3 0 0</cp>
138     <forward>0 1 0</forward>
139     <upward>0 0 1</upward>
140     <link_name>motor_4</link_name>
141 </plugin>
142 <!-- motor_f (ccw) lift-drag -->
143 <plugin filename="libLiftDragPlugin.so"
144     name="Motorf_1_LiftDrag">
145     <a0>0.76930</a0>
146     <alpha_stall>0.23</alpha_stall>
147     <cla>5.5286</cla>
148     <cda>0.0356</cda>
149     <cma>-0.0045</cma>
150     <cla_stall>-1.4670</cla_stall>
151     <cda_stall>1.0461</cda_stall>
152     <cma_stall>-0.050</cma_stall>
153     <area>0.041</area>
154     <air_density>1.2041</air_density>

```

```

155     <cp>-0.25 0 0</cp>
156     <forward>0 -1 0</forward>
157     <upward>0 0 -1</upward>
158     <link_name>motor_f</link_name>
159 </plugin>
160 <plugin filename="libLiftDragPlugin.so"
161     name="Motorf_2_LiftDrag">
162     <a0>0.76930</a0>
163     <alpha_stall>0.23</alpha_stall>
164     <cla>5.5286</cla>
165     <cda>0.0356</cda>
166     <cma>-0.0045</cma>
167     <cla_stall>-1.4670</cla_stall>
168     <cda_stall>1.0461</cda_stall>
169     <cma_stall>-0.050</cma_stall>
170     <area>0.041</area>
171     <air_density>1.2041</air_density>
172     <cp>0.25 0 0</cp>
173     <forward>0 1 0</forward>
174     <upward>0 0 -1</upward>
175     <link_name>motor_f</link_name>
176 </plugin>
177
178 <!-- main_wing lift-drag -->
179 <!-- Edited 10th June 2024 -->
180 <plugin filename="libLiftDragPlugin.so"
181     name="Left_MainWing_LiftDrag">
182     <a0>0.075</a0>
183     <cla>4.74394949633103</cla>
184     <cda>0.0684</cda>
185     <cma>-0.2172</cma>
186     <alpha_stall>0.23247786</alpha_stall>
187     <cla_stall>-0.8529</cla_stall>
188     <cda_stall>0.8918</cda_stall>
189     <cma_stall>-0.2172</cma_stall>
190     <cp>0.170968049 1.5 0.275</cp>
191     <area>2.1175</area>
192     <air_density>1.2041</air_density>
193     <forward>1 0 0</forward>
194     <upward>0 0 1</upward>
195     <link_name>base_link</link_name>
196     <control_joint_name>left_aileron_joint</control_joint_name>
197     <control_joint_rad_to_cl>-2.143</control_joint_rad_to_cl>
198 </plugin>
199
200 <plugin filename="libLiftDragPlugin.so"
201     name="Right_MainWing_LiftDrag">
202     <a0>0.075</a0>
203     <cla>4.74394949633103</cla>
204     <cda>0.0684</cda>
205     <cma>-0.2172</cma>
206     <alpha_stall>0.23247786</alpha_stall>
207     <cla_stall>-0.8529</cla_stall>
208     <cda_stall>0.8918</cda_stall>
209     <cma_stall>-0.2172</cma_stall>
210     <cp>0.170968049 -1.5 0.275</cp>
211     <area>2.1175</area>
212     <air_density>1.2041</air_density>
213     <forward>1 0 0</forward>
214     <upward>0 0 1</upward>
215     <link_name>base_link</link_name>
216     <control_joint_name>right_aileron_joint</control_joint_name>
217     <control_joint_rad_to_cl>-2.143</control_joint_rad_to_cl>
218 </plugin>
219

```

```

220 <!-- Aileron Aerodynamics Incorporated in Main Wings -->
221
222 <!-- hstab LiftDrag -->
223 <!-- Consider incorporating Elevator Joint -->
224 <!-- Edited 11th June 2024 -->
225
226 <plugin filename="libLiftDragPlugin.so"
227     name="hstab_LiftDrag">
228     <a0>-0.02</a0>
229     <cla>5.5992</cla>
230     <cda>0.03417112299</cda>
231     <cma>-0.07</cma>
232     <alpha_stall>0.3391428111</alpha_stall>
233     <cla_stall>-6.6621</cla_stall>
234     <cda_stall>1.9363984055</cda_stall>
235     <cma_stall>1.01</cma_stall>
236     <cp>-1.904031951 0 0.593</cp>
237     <area>1.013</area>
238     <air_density>1.2041</air_density>
239     <forward>1 0 0</forward>
240     <upward>0 0 1</upward>
241     <link_name>base_link</link_name>
242     <control_joint_name>elevator_joint</control_joint_name>
243     <control_joint_rad_to_cl>-3.0</control_joint_rad_to_cl>
244 </plugin>
245
246 <!-- vstab LiftDrag -->
247 <!-- Consider incorporating Rudder Joint -->
248 <!-- Edited on 11th June -->
249
250 <!-- left_vstab lift-drag -->
251 <plugin filename="libLiftDragPlugin.so"
252     name="left_vstab_LiftDrag">
253     <a0>0.0</a0>
254     <cla>5.5992</cla>
255     <cda>0.03417112299</cda>
256     <cma>-0.07</cma>
257     <alpha_stall>0.3391428111</alpha_stall>
258     <cla_stall>-6.6621</cla_stall>
259     <cda_stall>1.9363984055</cda_stall>
260     <cma_stall>1.01</cma_stall>
261     <cp>-1.904031951 1.15 0.244</cp>
262     <area>0.215</area>
263     <air_density>1.2041</air_density>
264     <forward>1 0 0</forward>
265     <upward>0 1 0</upward>
266     <link_name>base_link</link_name>
267     <control_joint_name>left_rudder_joint</control_joint_name>
268     <control_joint_rad_to_cl>-3.0</control_joint_rad_to_cl>
269 </plugin>
270 <!-- right_vstab lift-drag -->
271 <plugin filename="libLiftDragPlugin.so"
272     name="right_vstab_LiftDrag">
273     <a0>0.0</a0>
274     <cla>5.5992</cla>
275     <cda>0.03417112299</cda>
276     <cma>-0.07</cma>
277     <alpha_stall>0.3391428111</alpha_stall>
278     <cla_stall>-6.6621</cla_stall>
279     <cda_stall>1.9363984055</cda_stall>
280     <cma_stall>1.01</cma_stall>
281     <cp>-1.904031951 -1.15 0.244</cp>
282     <area>0.215</area>
283     <air_density>1.2041</air_density>
284     <forward>1 0 0</forward>

```

```
285     <upward>0 -1 0</upward>
286     <link_name>base_link</link_name>
287     <control_joint_name>right_rudder_joint</control_joint_name>
288     <control_joint_rad_to_cl>-3.0</control_joint_rad_to_cl>
289 </plugin>
290 <!-- End of the code -->
```